Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Direction Générale des Etudes Technologiques
Institut Supérieur des Etudes Technologiques de Mahdia

Support de révision

Le Langage C

Élaboré par :

Dr. Mohamed SBAI

NOTIONS DE BASE

Les programmes en C sont composés essentiellement de fonctions et de variables.

Définition d'une fonction en C

```
|<Type du résultat de la fonction> <Nom de la Fonction> (<Typepar1> <Nompar1>,
|                                                        <Typepar2> <Nompar2>,..)
|{
|<déclarations des variables locales>
|<instructions>
|}
```

Remarques

- ✓ En C, toute instruction simple est terminée par un point-virgule.
- ✓ En principe tout programme devrait retourner une valeur comme code d'erreur à son environnement.
- ✓ La fonction main est la fonction principale des programmes en C. Elle se trouve obligatoirement dans tous les programmes. L'exécution d'un programme entraine automatiquement l'appel de la fonction main.
- ✓ Si nous utilisons des fonctions prédéfinies (par exemple : printf), il faut faire précéder la fonction de main par l'appel des bibliothèques correspondantes en utilisant l'instruction #include (#include <stdio.h>) .
- ✓ Les commentaires

Un commentaire commence toujours par les deux symboles /* et se termine par les symboles */. Il est interdit d'utiliser des commentaires imbriqués.

Exemples

```
/* Ceci est un commentaire correct */
/*  Ceci est /* évidement */ défendu */
```

Les types entiers

Type	Description	Valeur min	Valeur max	Nombre d'octets
char	Caractère	-128	127	1
unsigned char	caractère non signé	0	255	1
short	entier court	-32768	32768	2
unsigned short	entier court non signé	0	65535	2
int	entier standard	-32768	32768	2
unsigned int	entier standard non signé	0	65535	2
long	entier long	-2147483648	2147483647	4
unsigned long	entier long non signé	0	4294967295	4

Les types rationnels

Les rationnels sont souvent appelés des flottants. Un rationnel est décrit comme suit :

<+/-> <**mantisse**>***10** <**exposant**>

Avec <+/-> est le signe positif ou négatif du nombre

< **mantisse**> est un décimal positif avec un seul chiffre devant la virgule.

<**exposant**> est un entier relatif

Type	Précision	Mantisse	Valeur min	Valeur max	Nombre d'octets
float	simple	6	$3.4*10^{-38}$	$3.4*10^{38}$	4
double	double	15	$1.7*10^{-308}$	$1.7*10^{308}$	8
Long double	suppl.	19	$3.4*10^{-4932}$	$3.4*10^{4932}$	10

Quelques constants caractères :

Caractère	Description
\n	Passage à la ligne
\t	Tabulation
\b	Curseur arrière
\r	Retour au début de la ligne
\a	Signal acoustique

Opérateurs arithmétiques

+	Addition
-	Soustraction
*	Multiplication
/	Division entière et rationnelle !)
%	Modulo

Opérateurs relationnels :

>	strictement supérieur
>=	supérieur ou égal
<	strictement inférieur
<=	inférieur ou égal
==	Egal
!=	Différent

Attention à ne pas confondre l'opérateur de test d'égalité "==" avec l'opérateur d'affection "=".

Opérateurs logiques booléens :

&&	ET logique
||	OU logique
!	négation logique

Opérateurs logiques sur les entiers bit à bit :

&	ET
|	OU
^	OU exclusif
~	Complément à 1 bit à bit
<<	Décalage à gauche
>>	Décalage à droite

Opérateurs d'affectation

Nous retrouvons souvent des affectations comme : i = i + 2
En C, nous utiliserons plutôt la formulation plus compacte : i += 2
Ceci est valable pour d'autres opérateurs détaillés dans ce tableau récapitulatif

+=	Ajouter à
-=	Dimimuer de
*=	Multiplier par
/=	Diviser par

Opérateurs d'incrémetation et de décrémentation

Les affectations les plus fréquentes sont du type :

I = I + 1 et I = I - 1

On peut utiliser ++ ou -- mais on doit distinguer les cas détaillés dans ce tableau

X = i++	Passe d'abord la valeur de i à x et incrémente après
X = ++i	Incrémente d'abord et passe la valeur incrémentée à x
X= i--	Passe d'abord la valeur de i à x et décrémente après
X= --i	Décrémente d'abord et passe la valeur décrémentée à x

Les priorités des opérateurs

Nous pouvons distinguer les classes de priorités suivantes :

Priorité 1 (la plus forte)	( )
Priorité 2	! ++ --
Priorité 3	* / %
Priorité 4	+ -
Priorité 5	< <= > >=
Priorité 6	== !=
Priorité 7	&&
Priorité 8	||
Priorité 9 (la plus faible)	= += -= *= /= %=

Les fonctions arithmétiques standard

Les fonctions suivantes sont prédéfinies dans la bibliothèque standard <math>. Pour pouvoir les utiliser, le programme doit contenir la ligne : #include<math.h>.

Remarque : les arguents et les résultats des fonctions arithmétiques sont du type double.

Commande en C	**Explication**
exp(x)	Fonction exponentielle
log(x)	Logarithme naturel
log10(x)	Logarithme à base 10
pow(x,y)	x exposant y
sqrt(x)	Racine carrée de x
fabs(x)	Valeur absolue de x
floor(x)	Arrondi en moins
ceil(x)	Arrondi en plus
fmod(x)	Reste rationnel de x/y
sin(x), cos(x), tan(x)	sinus, cosines, tangente de x

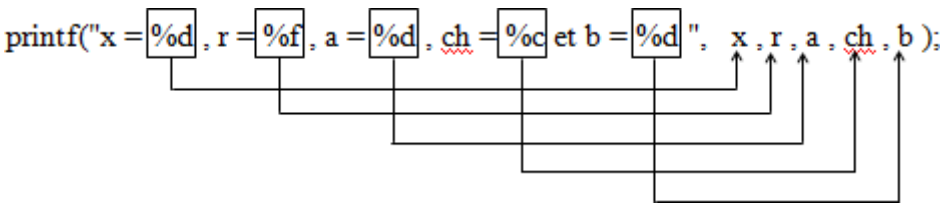
LIRE ET ECRIRE DES DONNEES

La bibliothèque standard < stdio> (aussi < conio>) contient un ensemble de fonctions permettant la communication entre la machine et le monde extérieur. Nous citons par exemple : printf ( ) , scanf( ), putchar( ) , getchar

Exemple printf : printf("La valeur de x est %d ", x) ;
Message Format Paramètre

Affichage multiple de variables :

printf("%format1 %format2 .... %formatn", variable1, variable2, ..., variablen);

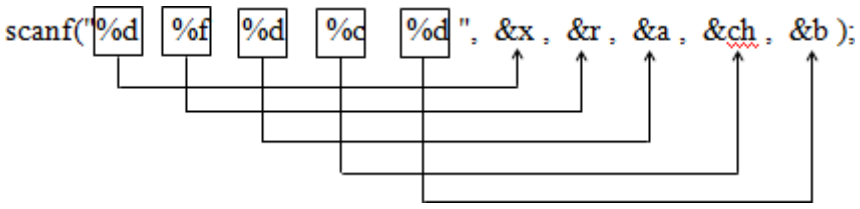


Exemple scanf : scanf(" %d ", &x);

Format Paramètre

Saisit multiple de variables :

scanf("%format1 %format2 .... %formatn", &variable1, &variable2, ..., &variablen);



Différents formats utilisés pour scanf et printf

Symbole	Impression comme (pour printf) et lecture d'un (e) (pour scanf)
%d ou % i	Entier
%u	Entier non signé
%o	Entier exprimé en octal
%x	Entier exprimé en hexadécimal
%c	Caractère
%s	Chaine de caractère
%f ou %e	Réel (float)

LES INSTRUCTIONS DE BRANCHEMENT ET LES BOUCLES

✓ L'instruction IF ... ELSE

Syntaxe :

```
if(expression)
      {
        .....;  /*bloc1d'instructions*/
        ..... ;
      }
      else
      {
        .....;  /*bloc2d'instructions*/
        . .....;
      }
```

Remarque : Les {} ne sont pas nécessaires lorsque les blocs ne comportent qu'une seule instruction.

✓ L'opérateur conditionnel "?"

L'opérateur conditionnel "?" est équivalent à l'instruction if else. Sa syntaxe est la suivante :

condition ? expression_1 : expression_2

Cette expression est égale à expression_1 si condition est satisfaite, et à expression_2 sinon.

Exemple : recherche du maximum de deux nombres entiers

```
if (a > b)  Max = a;          équivalent à     Max = (a > b ? a : b);
else        Max = b;
```

✓ L'instruction SWITCH

Sa forme la plus générale est celle-ci :

```
switch (expression)
{
   case constante-1 : liste d'instructions 1; break;
   case constante-2 : liste d'instructions 2; break;
   ...
   case constante-n : liste d'instructions n; break;
   default : liste d'instructions;
            }
```

Remarque : N'oubliez pas le break à la fin de chaque case dans une instruction switch. Si le break est absent, l'exécution se poursuit dans le case suivant.

✓ Boucle FOR

```
for (exp 1 ; exp 2 ; exp 3)
    {
    .....;          /* bloc d'instructions */
    ..... ;
    }
```

Avec : exp1 correspond à l'initialisation d'une variable de contrôle.

- exp2 est la condition de continuation de la boucle.
- exp3 est l'incrémentation de la variable de contrôle (elle est effectuée après l'exécution de l'instruction).

✓ Boucle Do… While

Algorithmique	*Langage C*
Répéter	**Do**
bloc d'instructions	{
	.....; /* bloc d'instructions */
jusqu'à (condition)	..... ;
	while (condition);

Remarque : Le bloc d'instruction sera exécuté tant que la condition est vérifiée (ou non nulle). Le test se faisant après, le bloc est exécuté au moins une fois même si l'expression est nulle à la première évaluation.

✓ Boucle While

Algorithmique	*Langage C*
tant que (condition) faire	**while** (condition)
	{
bloc d'instructions	.....; /* bloc d'instructions */
	. .....;
fin tant que	. .....;
	}

Tant que la condition est vérifiée (ou non nulle), le bloc d'instruction est exécuté. Le test ayant lieu avant l'exécution de l'instruction, celle-ci peut ne pas être exécutée si l'expression est nulle à la première évaluation.

Remarque :

- Les {} ne sont pas nécessaires lorsque le bloc ne comporte qu'une seule instruction.

for(expr 1 ; expr 2 ; expr 3)
instructions ;
	≡	expr 1 ;
		while (expr 2)
		{
		instructions ;
		expr 3 ;
		}
			≡	expr 1 ;
				do
				{
				instructions ;
				expr 3 ;
				}
				while (expr 2) ;

LES TABLEAUX

Les tableaux à une dimension

Déclaration

type nom[dim]

- type : définit le type d'élément que contient le tableau. Un tableau en langage C est composé uniquement d'éléments de même type.
- nom: est le nom du tableau.
- dim : est un nombre entier qui détermine le nombre des éléments du tableau.

Exemple:

```
Int tab[10]; /* le compilateur réserve des places pour 10 entiers, soit 40 octets */
float tableau2[20]; /* le compilateur réserve des places pour 20 réels, soit 80 octets */
```

Remarque:

dim est nécessairement une VALEUR NUMERIQUE.

Accéder aux éléments d'un tableau

Pour accéder à un élément du tableau, il suffit de donner le nom du tableau, suivi de l'indice de l'élément entre crochets :

```
nom[indice]
```

Exemples:

```
compteur[2] = 5;
nombre[i] = 6.789;
printf("%d",compteur[i]);
scanf("%f",&nombre[i]);
```

Remarques:

- L'indice du premier élément du tableau est 0.
- L'indice est toujours positif.
- L'indice du dernier élément du tableau est égal (dim– 1).

Les tableaux à plusieurs dimensions

Tableaux à deux dimensions

Déclaration

type nom[dim1][dim2]

Exemples:

```
int compteur[4][5];
float nombre[2][10];
```

Un élément du tableau est repéré par ses indices. En langage C, les tableaux commencent aux indices 0. Les indices maximum sont donc (dim1-1) et (dim2-1).

Exemples :

```
int tab[3][4]     /* 3 lignes et 4 colonnes */
tab[0][0] = 5;
printf("%d", tab[i][j]);
scanf("%f", &tab[i][j]);
```

Tableau[0][0]	Tableau[0][1]	Tableau[0][2]	Tableau[0][3]
Tableau[1][0]	Tableau[1][1]	Tableau[1][2]	Tableau[1][3]
Tableau[2][0]	Tableau[2][1]	Tableau[2][2]	Tableau[2][3]

Initialisation des tableaux

On peut initialiser les tableaux au moment de leur déclaration:

Exemples:

```
float nombre[4] = {2.67, 5.98 ,-8 ,0.09};
/* 2 lignes et 3 colonnes */
int x[2][3] = { {1, 5 ,7} , {8 ,4 ,3} };
```

Manipulation des tableaux

Pour la manipulation des tableaux, on utilise les boucles (for ou while ou do … while).

Saisie et affichage d'un tableau

Ecrire un programme permettant de saisir un tableau de réel de taille 10 :

Boucle for

```
main()
{
float nombre[10];
int i;
/* saisie du tableau*/
for( i=0; i<10; i++ )
        {
        printf("nombre[%d] = ", i);
        scanf("%f", &nombre[i]);
        }
/* affichage du tableau*/
for( i=0; i<10; i++ )
    printf("nombre[%d] = %f ",i,nombre[i]);
}
```

Boucle while

```
main()
{
float nombre[10];
int i;
/* saisie du tableau*/
while( i<10 )
        {
        printf("nombre[%d] = ", i);
        scanf("%f", &nombre[i]);
        i++ ;
        }
/* affichage du tableau*/
while( i<10 )
    {
    printf("nombre[%d] = %f ",i,nombre[i]);
    i++;
    } }
```

Exemple : Calcul de la moyenne d'un tableau

Ecrire un programme permettant de saisir un tableau de 10 entiers, de calculer et d'afficher leur moyenne.

```
int tab[10];
int  i,s;
float moy;
/*initialisation de la somme et la moyenne*/
s =0 ;
moy =0.0 ;
/* saisie du tableau*/
for(i=0;i<10;i++)
{
        printf("tab[%d] = ",i);
        scanf("%d", &tab[i]);
}
/* calcul de la moyenne */
for(i=0; i<10; i++)
        s = s+ tab[i];

moy = s / 10;
/* affichage de la moyenne */
printf("Moyenne du tableau = %f", moy);
// ou bien   printf("Moyenne du tableau = %f", float(s/10) );
```

Exemple : Rechercher le maximum d'un tableau

Ecrire un programme permettant de saisir la moyenne de 30 étudiants, de calculer et d'afficher la plus grande moyenne.

```
float moy[30];
int i;
float max;

/* saisie du tableau*/
for(i=0; i<10; i++)
{
        printf("moy[%d] = ",i);
        scanf("%f", &moy[i]);
}
max = moy[0] ;
/* recherche du maximum */
for(i=0; i<10; i++)
        if (moy[i] > max)
            max = moy[i];

/* affichage du maximum */
printf("Maximum du tableau = %f", max);
```

Exemple : Saisir et afficher un tableau à 2 dimensions :

Ecrire un programme permettant de saisir un tableau de réel de taille 5x10 :

```
float nombre[5][10];
int i,j;
/* saisie du tableau*/
for( i=0; i<5; i++ )
        for( j=0; j<10; j++ )
        {
                printf("nombre[%d][%d] = ", i,j);
                scanf("%f", &nombre[i][j]);
        }

/* affichage du tableau*/
for( i=0; i<5; i++ )
{
          for( j=0; j<10; j++ )
                printf("nombre[%d][%d] = %f ", i, j, nombre[i][j] );
          printf("\n");
}
```

LES CHAINES DE CARACTERES

Déclaration

Une chaîne est un tableau de caractères dont le dernier élément significatif est le caractère '**\0**'. Il faut donc différencier la longueur maximale de la chaîne (c'est à dire la dimension donnée lors de la définition) de sa longueur réelle (c'est à dire le nombre de caractères jusqu'au '\0').

char nom[dim]

Exemple :

```
char texte[dim];
```

Le compilateur réserve (dim-1) places en mémoire pour la chaîne de caractères. En effet, il ajoute toujours le caractère NUL ('\0') à la fin de la chaîne en mémoire.

Exemples :

```
char chaine[] = "Hello";
```

H	E	l	l	o	\0

```
char chaine[6] = "Hello";
```

H	e	l	l	o	\0

```
char chaine[8] = "Hello";
```

H	e	l	l	o	\0	0	0

```
char chaine[5] = "Hello";
```

H	e	l	l	o

==> **Erreur pendant l'exécution**

```
char chaine[4] = "Hello";
```

==> **Erreur pendant la compilation**

Affichage

On peut utiliser la fonction **printf** avec le format **%s** :

```
char texte[10] = "BONJOUR";
printf("Voici le texte: %s\n", texte);
```

On peut aussi utiliser la fonction puts :

```
puts(texte);        ≡        printf("%s\n",texte);
```

Saisit

On peut utiliser la fonction **scanf** avec le format le format **%s**. On n'écrit pas le symbole **&**. On peut aussi utiliser la fonction **gets**.

```
char texte[10];
printf("entrer un texte: ");
scanf("%s", texte);        ≡        gets(texte);
```

Remarques :

- La fonction scanf ne permet pas la saisie d'une chaîne comportant des espaces : les caractères saisis à partir de l'espace ne sont pas pris en compte. Pour saisir une chaîne comportant des espaces, il faut utiliser la fonction gets.
- A l'issue de la saisie d'une chaîne de caractères, le compilateur ajoute '\0' en mémoire après le dernier caractère.

Exemples :

```
char lieu[25];
int jour, mois, annee;
printf("Entrez lieu et date de naissance :");
scanf("%s %d %d %d", lieu, &jour, &mois, &annee);
```

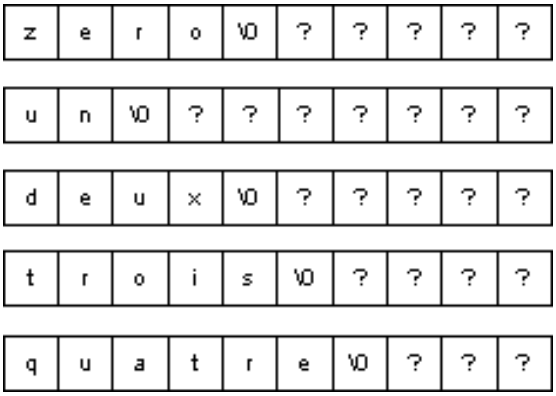
Tableau de chaînes de caractères

Les tableaux de chaînes de caractères sont des tableaux de tableaux de caractères.

Exemple :

```
char tab[5][10] = { "zero", "un", "deux", "trois", "quatre" };  /* 5 lignes et 10 colonnes */
```

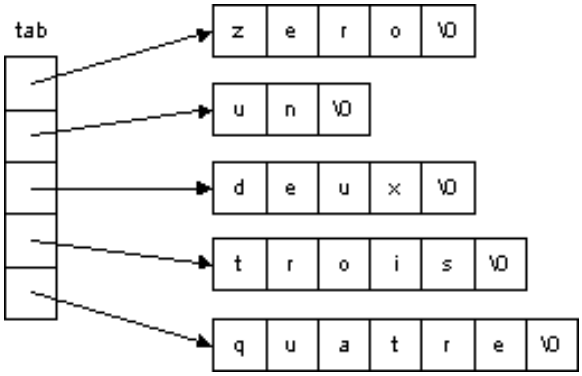
Ce qui donne:



On constate que le tableau contient de l'espace inutilisé relativement important. Pour minimiser la mémoire nécessaire au stockage des chaînes, on peut déclarer **tab** comme un tableau de pointeurs :

```
char *tab[5] = { "zero", "un", "deux", "trois", "quatre" };
```

Ce qui donne :



Fonctions permettant la manipulation des chaînes

Les bibliothèques fournies avec les compilateurs contiennent de nombreuses fonctions de traitement des chaînes de caractères. En C++, elles appartiennent aux bibliothèques string.h ou stdlib.h. En voici quelques exemples:

String.h :

strlen(s)	Fournit la longueur de la chaîne sans compter le '\0' final
strcpy(s, t)	Copie <t> vers <s>
strcat(s, t)	Ajoute <t> à la fin de <s>
strcmp(s, t)	Compare <s> et <t> et fournit un résulta t : négatif si <s> précède <t> zéro si <s> est égal à <t> positif si <s> suit <t>
strncpy(s, t, n)	Copie au plus <n> caractères de <t> vers <s>
strncat(s, t, n)	Ajoute au plus <n> caractères de <t> à la fin de <s>
strchr(s, c)	Trouve la première occurrence du caractère c dans la chaîne s, c'est à dire la première fois où il apparaît
strrev (s)	Inversion de l'ordre de la chaîne s
strupr (s)	Conversion des lettres minuscules en lettres majuscules de la chaîne s

LES STRUCTURES

Définition et déclaration

Le langage C autorise la déclaration de types particuliers: les structures. Une structure est constituée de plusieurs éléments ou champs de même ou de différent type.

```
typedef struct
{
        type1 nom_champ1 ;
        type2  nom_champ2 ;
        . . .
        typeN  nom_champ_N ;
} nom_structure ;
```

Exemple :

```
typedef struct                          /* On définit un type struct */
{
        char nom[10];
        char prenom[10];
        int age;
        float note;
}  etudiant;
```

Cette structure définit les champs nom et prénom de type char, age de type int et note de type float.

Déclaration :

On déclare des variables par exemple : fiche e1, e2;

Utilisation:

```
strcpy(e1.nom, "ALI");
strcpy(e1.prenom, "Mohamed");
e1.age = 20;
e1.note = 11.75;
```

Rq :

L'affectation globale est possible avec les structures : e2 = e1;

Par contre, on ne peut pas comparer deux structures. Il faut les comparer champs par champs.

~~if (e1 = = e2) .....~~

Exercice 1 :

Créer la structure etudiant ci-dessus, saisir un etudiant puis l'afficher.

Corrigé

```
#include<stdio.h>

typedef struct
{
        char nom[10];
        char prenom[10];
        int age;
        float note;
```

```
} etudiant;

main()
{
        etudiant f;
        printf("\n\nSaisir une fiche :\n");
        printf("Nom : ");
        gets(f.nom);
        printf("Prenom : ");
        gets(f.prenom);
        printf("Age : ");
        scanf("%d", &f.age);
        printf("Note : ");
        scanf("%f", &f.note);
        printf("\n\nAffichage de la fiche :\n");
        printf("Nom:%s Prenom:%s ", f.nom, f.prenom,);
        printf("Age:%d Note:%f", f.age, f.note);
}
```

Structures et tableaux

On peut définir un tableau de structures :

Déclaration :

```
etudiant e[10];          /* on déclare un tableau de type etudiant de taille 10 */
                         /* etudiant est la structure définie précédemment */
```

Utilisation :

```
strcpy(e[i].nom,"Ali")          /* pour un indice i quelconque */
strcpy(e[i].prenom,"Mohamed");
e[i].age = 20;
e[i].note = 11.75;
```

Exercice 2 :
Créer une structure note{char nom[20] ; float d; float e;}
Saisir un tableau contenant 4 notes puis l'afficher.

Corrigé

```
#include <stdio.h>

typedef struct
{
        char nom[20];
        float d;
        float e;
} note;

main()
{
        note p[4];                              /* tableau de note */
        int i;
        printf("Saisie des notes\n");           /* saisie */
        for(i=0; i<4; i++)
        {
```

```
                printf("\nDonnez le nom de l'etudiant n:d\n", i);
                gets(p[i].nom);
                printf("Devoir= ");
                scanf("%f", &p[i].d);
                printf("Examen= ");
                scanf("%f", &p[i].e);
        }
        printf("\nAffichage\n");                          /* affichage */
        for(i=0; i<4; i++)
        {
                printf("\nEtudiant n%d : %s", i, p[i].nom);
                printf("\nDevoir= %f", p[i].d);
                printf("\nExamen= %f\n", p[i].e);
        }
}
```

Structure de structure

On peut utiliser une structure comme champ d'une autre structure.

Exemple :

On peut définir une structure adresse (contenant numéro et l'adresse d'une maison) qui pourra être utilisée dans la structure repertoire (contenant les coordonnées d'une personne : nom, prénom et adresse). Elle peut également être utilisée dans une autre structure.

```
typedef struct
{
        int numero ;
        char rue[50] ;
} adresse ;

typedef struct
{
        char nom[20] ;
        char prenom[20] ;
           adresse maison ;              /* déclaration d'un champ structure */
 } repertoire ;                          /* de type 'adresse' appelé 'maison' */
```

Utilisation :

```
repertoire monrepertoire[100] ;

strcpy(monrepertoire[0].nom , "Salah") ;
strcpy(monrepertoire[0].prenom , "Mohamed") ;
monrepertoire[0].maison.numero = 19 ;
monrepertoire[0].maison.rue = "avenue_7_novembre" ;
strcpy(monrepertoire[1].nom , "Abid") ;
strcpy(monrepertoire[1].prenom , "Ali") ;
monrepertoire[1].maison.numero = 15 ;
monrepertoire[1].maison.rue = "rue_menzel_chaker" ;
```

Lorsqu'un tableau fait partie des champs d'une structure, on peut accéder aux valeurs de ce tableau par :

```
char initiale ;
initiale = monrepertoire[1].prenom[0] ;          /* initiale de Ali */
```

Exercice 1 :

Créer une structure **etudiant** contenant 3 champs :

- nom de type chaîne de caractère.
- prenom de type chaîne de caractère.
- moyen de type réel.

Créer une autre structure **classe** contenant 3 champs :

- Filière de type chaîne de caractère (TIT ou TET ou MSI).
- Niveau de type entier (1 ou 2 ou 3).
- Eleve de type **etudiant**

Ecrire un programme permettant de saisir les noms, prénom et moyen d'une **classe** de 10 **étudiants**. Puis afficher les noms, prénom et moyen de cette classe.

Corrigé :

```
#include<stdio.h>
#include<conio.h>
#include<string.h>

typedef struct
{
        char nom[10];
        char prenom[10];
        float moyen;
} etudiant;

typedef struct
{
        char filiere[3];
        int niveau;
        etudiant eleve[10];
} classe;

main()
{
        classe c;
        int i;

        strcpy(c.filiere , "IT");
        c.niveau = 1;

        /* saisie d'une classe */
        printf("Saisie des etudiant de la classe\n");
        for(i=0; i<10; i++)
        {
```

```
                printf("\nDonnez le nom de l'etudiant n %d: ", i);
                gets(c.eleve[i].nom);
                printf("\nDonnez le prenom de l'etudiant n %d: ", i);
                gets(c.eleve[i].prenom);
                printf("\nDonnez le moyen de l'etudiant n %d: ", i);
                scanf("%f", &c.eleve[i].moyen);
        }

        /* affichage d'une classe */
        printf("\nAffichage des etudiant de la classe %d %s, \n", c.niveau, c.filiere) ;
        for(i=0; i<10; i++)
        {
                printf("Nom : %s", c.eleve[i].nom);
                printf("\tPrenom= %s", c.eleve[i].prenom);
                printf("\tMoyen= %f\n", c.eleve[i].moyen);
        }
}
```